

MINI PROJECT - 3

NAME: Nandhini V

DATE: 25-07-2025

Question:

Holiday Package

The tour package details like package Id, source place, destination place, basic fare and number of days for each package should be stored in a table. Calculate the package cost based on the basic fare and the number of days for each package.

Instance_variable	Data type
--------------------------	------------------

package_id	String
------------	--------

source_place	String
--------------	--------

destination_place	String
-------------------	--------

no_of_days	int
------------	-----

package_cost	double
--------------	--------

Create a model class to represent this with attributes, getters and setters.

Calculate the package cost for each package, based on the conditions given below,

noOfDays	discount %
----------	------------

<=5	0% (No discount)
-----	------------------

>5 and <=8	3%
------------	----

>8 and <=10	5%
-------------	----

>10	7%
-----	----

Package Cost = ((Basic fare x number of days)-discount) + GST

The package cost should be calculated based on the basic fare and the number of days. The discount should be calculated depending on the number of days as given in the above table and

deducted from the calculated package cost. Finally, a GST of 12% of the calculated package cost got after the discount, should be added to get the final package cost.

For example: If a package has a basic fare as Rs.3000 and the number of days as 15, then the package cost will be (3000×15) , which is Rs. 45000.00. Since the number of days is 15, the discount percentage will be 7%. So, the discount will be $(45000.0 \times (7/100))$ which is Rs. 3150.00. Now, 12% of GST needs to be added. So the GST will be $((45000.0 - 3150.0) \times (12/100))$ which is Rs. 5022.00.

Therefore, the total cost for this package will be $((3000 \times 15) - 3150.0) + 5022.0$ which is Rs. 46872.00.

Develop as a menu-driven application. This application should show options

1. Add Package details
2. Display all package details
3. Search for a package with package id
4. Calculate package cost based on package id

Use layered architecture (Client (Main class), Service Layer, Dao Layer)

Classes to be created:

Model class - Package –attributes (packageid, source, destination, no_of_days and package cost)

PackageDao interface – declare the methods for adding package details, displaying all packages, calculating package cost and searching a package from the collection.

PackageDaoImpl class – implement the method declared in PackageDao interface

PackageService interface – declare the methods – addPackage, findPackageById, fetchAllPackages, calculatePackageCost

PackageServiceImpl class –implement the methods of PackageService and call the methods of PackageDao class.

Main class – Create object of service class and call the methods by getting choice from the user.

Validation:

The packageId should be validated before calculating the package cost; only if the packageId is valid, the Package object should be added to the list. The packageId should be in the following format.

The packageId should contain exactly 7 characters.

If the packageId is valid, then calculate the package cost, else throw a user defined Exception "InvalidPackageIdException" with a message "Invalid Package Id".

Code:

TourPackage.java - org.model

```
package org.model;
public class TourPackage {
    private String packageId;
    private String sourcePlace;
    private String destinationPlace;
    private int noOfDays;
    private double basicFare;
    private double packageCost;
    public TourPackage(String packageId, String sourcePlace, String destinationPlace, int
noOfDays, double basicFare) {
        this.packageId = packageId;
        this.sourcePlace = sourcePlace;
        this.destinationPlace = destinationPlace;
        this.noOfDays = noOfDays;
        this.basicFare = basicFare;
    }
    public String getPackageId() {
        return packageId;
    }
    public String getSourcePlace() {
        return sourcePlace;
    }
    public String getDestinationPlace() {
        return destinationPlace;
    }
    public int getNoOfDays() {
        return noOfDays;
    }
    public double getBasicFare() {
```

```

        return basicFare;
    }
    public double getPackageCost() {
        return packageCost;
    }
    public void setPackageCost(double packageCost) {
        this.packageCost = packageCost;
    }
    @Override
    public String toString() {
        return "\nPackageId: " + packageId + ",\nSource: " + sourcePlace +
",\nDestination: " + destinationPlace + ",\nDays: " + noOfDays + ",\nBasic Fare: " + basicFare +
",\nCost: " + packageCost;
    }
}

```

PackageDao.java - org.dao

```

package org.dao;
import org.model.TourPackage;
import java.util.*;
public interface PackageDao {
    void addPackage(TourPackage p);
    List<TourPackage> fetchAllPackages();
    TourPackage findPackageById(String packageId);
    void calculatePackageCost(TourPackage p);
}

```

PackageDaoImpl.java - org.dao

```

package org.dao;
import org.model.TourPackage;
import java.util.*;
public class PackageDaoImpl implements PackageDao {
    private List<TourPackage> packages = new ArrayList<>();
    public void addPackage(TourPackage p) {
        packages.add(p);
    }
    public List<TourPackage> fetchAllPackages() {
        return packages;
    }
    public TourPackage findPackageById(String packageId) {

```

```

        for (TourPackage p : packages) {
            if (p.getPackageId().equals(packageld))
                return p;
        }
        return null;
    }

    public void calculatePackageCost(TourPackage p) {
        double total = p.getBasicFare() * p.getNoOfDays();
        double discount = 0;
        if (p.getNoOfDays() > 10)
            discount = 0.07;
        else if (p.getNoOfDays() > 8)
            discount = 0.05;
        else if (p.getNoOfDays() > 5)
            discount = 0.03;
        double discountedAmount = total * discount;
        double gst = (total - discountedAmount) * 0.12;
        double cost = total - discountedAmount + gst;
        p.setPackageCost(cost);
    }
}

```

PackageService.java - org.service

```

package org.service;
import org.model.TourPackage;
import java.util.*;
public interface PackageService {
    void addPackage(TourPackage p) throws Exception;
    List<TourPackage> fetchAllPackages();
    TourPackage findPackageById(String id);
    void calculatePackageCost(String packageld) throws Exception;
}

```

PackageServiceImpl.java - org.service

```

package org.service;
import org.dao.*;
import org.model.TourPackage;
import org.exception.InvalidPackageIdException;
import java.util.*;
public class PackageServiceImpl implements PackageService {

```



```

        System.out.println("Enter Destination:");
        String dest = sc.nextLine();
        System.out.println("Enter No. of Days:");
        int days = sc.nextInt();
        System.out.println("Enter Basic Fare:");
        double fare = sc.nextDouble();
        TourPackage p = new TourPackage(id, source, dest, days, fare);
        service.addPackage(p);
        System.out.println("Package added.");
        break;
    case 2:
        for (TourPackage pkg : service.fetchAllPackages())
            System.out.println(pkg);
        break;
    case 3:
        System.out.println("Enter Package ID:");
        String searchId = sc.nextLine();
        TourPackage found = service.findPackageById(searchId);
        System.out.println(found != null ? found : "Package not found.");
        break;
    case 4:
        System.out.println("Enter Package ID:");
        String costId = sc.nextLine();
        service.calculatePackageCost(costId);
        System.out.println("Cost calculated.");
        break;
    case 5:
        System.out.println("Exiting.");
        sc.close();
        return;
    default:
        System.out.println("Invalid choice.");
    }
} catch (Exception e) {
    System.out.println(e.getMessage());
}
}
}
}

```

InvalidPackageIdException.java - org.exception

```
package org.exception;  
public class InvalidPackageIdException extends Exception {  
    public InvalidPackageIdException(String msg) {  
        super(msg);  
    }  
}
```

Output:

1. Add Package
2. Display All Packages
3. Search by ID
4. Calculate Cost by ID
5. Exit

1

Enter Package ID:

1234567

Enter Source:

cbe

Enter Destination:

blr

Enter No. of Days:

6

Enter Basic Fare:

700

Package added.

1. Add Package
2. Display All Packages
3. Search by ID
4. Calculate Cost by ID
5. Exit

2

PackageId: 1234567,

Source: cbe,

Destination: blr,

Days: 6,

Basic Fare: 700.0,

Cost: 0.0

1. Add Package
2. Display All Packages
3. Search by ID

4. Calculate Cost by ID
5. Exit

3

Enter Package ID:

1234567

Packageld: 1234567,

Source: cbe,

Destination: blr,

Days: 6,

Basic Fare: 700.0,

Cost: 0.0

1. Add Package
2. Display All Packages
3. Search by ID
4. Calculate Cost by ID
5. Exit

4

Enter Package ID:

1234567

Cost calculated.

1. Add Package
2. Display All Packages
3. Search by ID
4. Calculate Cost by ID
5. Exit

2

Packageld: 1234567,

Source: cbe,

Destination: blr,

Days: 6,

Basic Fare: 700.0,

Cost: 4562.88

1. Add Package
2. Display All Packages
3. Search by ID
4. Calculate Cost by ID
5. Exit

5

Exiting.